

외부 API 장애 대응 Decorator 패턴 + Circuit Breaker

Retry · Exponential Backoff · Jitter · NON_RETRYABLE · Circuit Breaker

M3 VASP 추상화

S17

Day 05

S17 아젠다

강의 20분 + 실습 35분

세션 구성

- § 1 재시도 전략
- § 2 Decorator 패턴
- § 3 Circuit Breaker
- § 4 실습

실습 목표

- [1] LoggingAdapterDecorator 구현
- [2] RetryAdapterDecorator 구현
- [3] NON_RETRYABLE 처리
- [4] Decorator 조합
- [5] CircuitBreakerService 구현

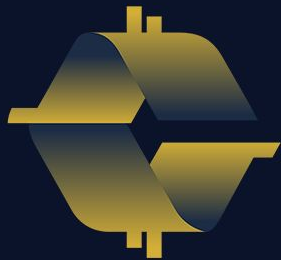
핵심 질문: 외부 VASP API가 일시적으로 장애가 났을 때, 어떻게 안전하게 재시도하고 장애를 차단할 것인가?

§ 1 재시도 전략

Exponential Backoff · Jitter · NON_RETRYABLE

Retry Storm

Thundering Herd



COINCRAFT
ACADEMY

즉각 재시도의 위험

§ 1 재시도 전략

Retry Storm: 장애 서비스에 트래픽이 집중되어 복구가 불가능해진다

시나리오

- VASP API 일시 장애 발생
- 1,000개 Consumer 동시 실패
- 모두 즉각 재시도 시작
- VASP에 1,000개 요청 동시 집중
- 복구 시도 중인 VASP 다시 다운

해결책 미리 보기

- Exponential Backoff — 재시도 간격 점진적 증가
- Jitter — 재시도 시점 무작위 분산
- NON_RETRYABLE — 불필요한 재시도 차단
- Circuit Breaker — 장애 서비스 요청 차단

즉각 재시도: 실패 → (0ms) → 재시도 → 실패 → (0ms) → 재시도 → ...

Backoff 재시도: 실패 → (1s) → 재시도 → 실패 → (2s) → 재시도 → ...

Exponential Backoff

§ 1 재시도 전략

공식: $\text{delay} = \min(\text{initialDelay} \times 2^{\text{attempt}}, \text{maxDelay})$

지수 증가 예시 (initialDelay = 1s, max = 30s)

- attempt 0 → 1s
- attempt 1 → 2s
- attempt 2 → 4s
- attempt 3 → 8s
- attempt 4 → 16s
- attempt 5 → 30s (cap)

코드

```
let delayMs = initialDelayMs;

for (let attempt = 1; attempt <= maxAttempts; attempt++) {
  try { return await fn(); }
  catch (err) {
    if (nonRetryable(err)) throw err;
    if (attempt === maxAttempts) throw err;

    const jittered = delayMs * (0.8 + Math.random() * 0.4);
    await sleep(jittered);
    delayMs = Math.min(delayMs * backoffFactor, maxDelayMs);
  }
}
```

의미: 실패할수록 더 오래 기다린다 → VASP가 복구할 시간을 확보한다

Jitter — Thundering Herd 방지

§ 1 재시도 전략

Thundering Herd: 여러 인스턴스가 동일한 Backoff 간격으로 동시에 재시도 → 결국 동시 집중

Jitter 적용 방법

- baseDelay에 $\pm 20\%$ 무작위 편차 추가
- 범위: $\text{baseDelay} \times 0.8 \sim \text{baseDelay} \times 1.2$
- 인스턴스마다 다른 시점에 재시도

```
const jittered =  
  delayMs * (0.8 + Math.random() * 0.4);  
await sleep(jittered);  
delayMs = Math.min(delayMs * backoffFactor, maxDelayMs);
```

효과 비교

Jitter 없음 (1,000 인스턴스):

t=2s → 1,000개 동시 재시도 💣

Jitter 적용:

t=1.6s → 50개

t=1.8s → 200개

t=2.0s → 500개

t=2.2s → 200개 ← 분산됨

효과: 재시도 시점이 시간축으로 분산 → VASP 부하가 고르게 분산된다

NON_RETRYABLE 패턴

§ 1 재시도 전략

원칙: NON_RETRYABLE 에러를 재시도하면 가스비만 낭비된다 — 즉시 throw하라

재시도 불필요 (즉시 throw)

- REVERT — 컨트랙트 조건 미충족
- 잔액 부족 — insufficient balance
- AccessControl 거절 — 권한 없음
- ERC1155: burn amount exceeds balance
- → 재시도해도 결과 동일

재시도 가능 (Backoff 적용)

- 네트워크 타임아웃
- 5xx 서버 에러
- 429 Rate Limit
- 일시적 RPC 연결 실패
- → 잠시 후 재시도하면 성공 가능

```
const NON_RETRYABLE = ['execution reverted', 'insufficient funds', 'nonce too high'];

const msg = String(err).toLowerCase();
if (NON_RETRYABLE.some(p => msg.includes(p))) throw err;
```

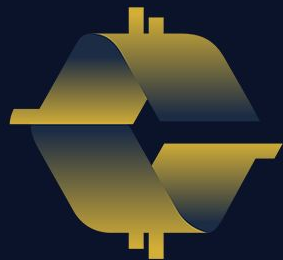
§ 2

Decorator 패턴

LoggingAdapterDecorator · RetryAdapterDecorator · 조합

관심사 분리

IBlockchainAdapter



COINCRAFT
ACADEMY

Decorator 패턴 구조

§ 2 Decorator 패턴

구조

- IBlockchainAdapter 인터페이스를 감싸는 래퍼
- 원본 어댑터 코드 변경 없음
- 기능을 레이어로 추가
- 조합 순서에 따라 실행 순서 결정

핵심: 원본 어댑터 코드 변경 없이 기능 추가

데코레이터 종류

- **LoggingAdapterDecorator** — 경과 시간 측정
- **RetryAdapterDecorator** — 일시적 실패 자동 재시도

```
// 조합: 재시도 + 로깅
new RetryAdapterDecorator(
    new LoggingAdapterDecorator(
        realAdapter
    )
)
```

호출 흐름: RetryDecorator → LoggingDecorator → RealAdapter → VASP

응답 흐름: VASP → RealAdapter → LoggingDecorator → RetryDecorator → 호출자

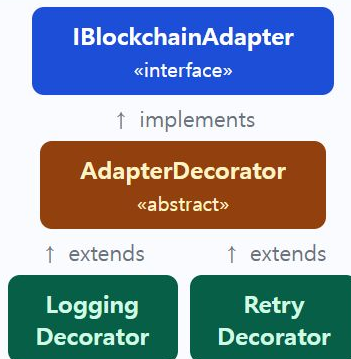
Decorator 3계층 구조

§ 2 Decorator 패턴

계층 역할

- **IBlockchainAdapter** — 인터페이스. 메서드 시그니처만 선언.
EVMAdapterDecorator 모두 이걸 구현 → 호출자는 누가 들어와도 동일하게 사용
- **AdapterDecorator** — abstract 베이스. inner를 받아 모든 메서드를 `this.inner.xxx()`로 위임. 서브클래스는 바꿀 메서드만 override
- **Logging / Retry** — 구현체. extends AdapterDecorator. 필요한 메서드만 override해서 앞뒤로 코드 끼워넣음

구조도



베이스 클래스가 필요한 이유: Logging·Retry 모두 "10개 메서드를 inner에 위임"하는 동일한 코드가 필요 → 베이스 클래스가 한 번만 작성, 서브클래스는 override만

LoggingAdapterDecorator

§ 2 Decorator 패턴

역할

- IBlockchainAdapter 모든 메서드 래핑
- 메서드 호출 전 Date.now() 기록
- 메서드 호출 후 elapsed ms 계산
- logs 배열에 기록
- 느린 RPC 탐지 · SLA 모니터링

구현 예시

```
async mintNFT(p: MintParams) {  
  const start = Date.now();  
  const result =  
    await this.inner.mintNFT(p);  
  const elapsed = Date.now() - start;  
  console.log(  
    `mintNFT elapsed: ${elapsed}ms`  
  );  
  return result;  
}
```

활용: 느린 RPC 탐지 → SLA 기준 초과 시 알림 → 어댑터 교체 판단 근거

RetryAdapterDecorator

§ 2 Decorator 패턴

재시도 로직

- maxAttempts: 기본 3회
- NON_RETRYABLE 에러 → 즉시 throw
- 재시도 가능 에러 → Exponential Backoff + Jitter
- maxAttempts 초과 → 마지막 에러 rethrow

핵심 코드

```
for (let i = 0; i < maxAttempts; i++) {  
  try {  
    return await this.inner.mintNFT(p);  
  } catch (err) {  
    if (isNonRetryable(err)) throw err;  
    if (i === maxAttempts - 1) throw err;  
    await sleep(calcJitteredDelay(i));  
  }  
}
```

장점: 재시도 로직이 어댑터 내부가 아닌 데코레이터에 위치 → 어댑터 교체 시 재시도 로직 그대로 재사용

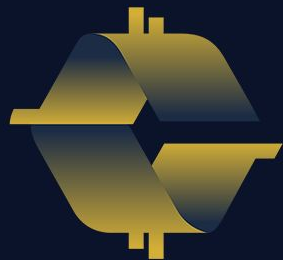
§ 3

Circuit Breaker

CLOSED · OPEN · HALF_OPEN 상태 전이

장애 차단

CircuitOpenError



COINCRAFT
ACADEMY

Circuit Breaker 3가지 상태

§ 3 Circuit Breaker

CLOSED — 정상

- 모든 요청 통과
- 실패 카운터 누적
- 임계치 미달 시 유지
- 연속 5회 실패 → OPEN

OPEN — 차단

- 모든 요청 즉시 거절
- CircuitOpenError 발생
- VASP에 요청 미전송
- 30s 후 → HALF_OPEN

HALF_OPEN — 탐색

- 탐색 요청 1개 허용
- 성공 → CLOSED 복귀
- 실패 → 다시 OPEN
- VASP 복구 여부 확인



목적: 장애 서비스에 대한 요청을 아예 차단해 복구 시간을 확보한다

Circuit Breaker 전이 상세

§ 3 Circuit Breaker

파라미터	기본값	의미
<code>failureThreshold</code>	5	연속 5회 실패 시 OPEN 전이
<code>cooldownMs</code>	30,000	OPEN 후 30초 경과 시 HALF_OPEN 전이
성공 시	reset	연속 실패 카운터 리셋 → CLOSED 유지
<code>CircuitOpenError</code>	즉시 throw	"Circuit is OPEN, request rejected"

OPEN 상태: 시도조차 하지 않는다 — VASP에 요청을 보내지 않고 즉시 에러를 반환해 복구 시간을 확보한다

HALF_OPEN 전이 조건: 마지막 실패로부터 `cooldownMs` 경과 후 첫 번째 요청이 들어올 때 자동 전이

§ 4

실습

npm run exercise:s17:answer

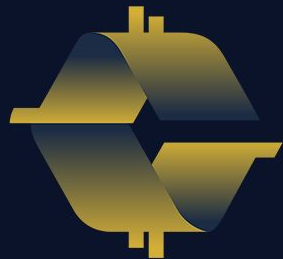
[1]
Logging

[2]
Retry

[3]
NON_RETRYABLE

[4]
조합

[5]
Circuit
Breaker



COINCRAFT
ACADEMY

실습 [1][2] Logging + Retry

§ 4 실습

[1] LoggingAdapterDecorator

- 경과 시간 측정 → logs 배열에 기록
- 형식: `mintNFT elapsed: Xms`
- 원본 결과값 그대로 반환
- 테스트: `logs.length === 1` 확인

[2] RetryAdapterDecorator

- stub: `failUntil=2` 설정
- → 1, 2번째 시도: 실패
- → 3번째 시도: 성공
- 기대: `callCount=3`, 성공 txHash 반환
- Backoff 적용: 1s → 2s 대기

확인 포인트: Retry 데코레이터가 내부적으로 Exponential Backoff + Jitter를 적용해 재시도하는지 확인

실습 [3][4] NON_RETRYABLE + 조합

§ 4 실습

[3] NON_RETRYABLE 처리

- 에러: `ERC1155: burn amount exceeds balance`
- `isNonRetryable()` → true 반환
- 재시도 없이 즉시 throw
- 기대: `callCount=1` (재시도 없음)
- 에러 메시지 그대로 전파

[4] Decorator 조합

```
const adapter =  
  new RetryAdapterDecorator(  
    new LoggingAdapterDecorator(  
      stub  
    )  
  );
```

- 로깅 + 재시도 동시 적용
- logs에 경과 시간 기록됨
- 실패 시 자동 재시도

조합의 의미: 각 데코레이터는 단일 책임만 가짐 → 조합으로 복합 기능 구현, 원본 코드 무변경

실습 [5] Circuit Breaker

§ 4 실습

테스트 시나리오

- 초기 상태: CLOSED
- 연속 5회 실패 → OPEN 전이
- OPEN 상태에서 요청 → CircuitOpenError
- 30s 쿨다운 (mock: forceHalfOpen)
- HALF_OPEN에서 성공 → CLOSED 복귀

상태 확인 코드

```
// 초기 상태
expect(svc.getState()).toBe('CLOSED');

// 5회 실패 후
expect(svc.getState()).toBe('OPEN');

// 쿨다운 후 탐색 성공
expect(svc.getState()).toBe('CLOSED');
```

CLOSED → (5회 실패) → OPEN → (forceHalfOpen) → HALF_OPEN → (성공) → CLOSED

핵심 정리

S17 — Decorator 패턴 + Circuit Breaker

패턴	목적	핵심
Exponential Backoff	VASP 복구 시간 확보	$delay = initialDelay \times 2^n$
Jitter	Thundering Herd 방지	$delay \times (0.8 + rand \times 0.4)$
NON_RETRYABLE	재시도 불필요 에러 즉시 처리	REVERT / 잔액부족 → 즉시 throw
Decorator 조합	관심사 분리	<code>Retry(Logging(adapter))</code>
Circuit Breaker	장애 서비스 차단	CLOSED → OPEN → HALF_OPEN → CLOSED

M3 핵심 원칙: 외부 의존성 장애에 흔들리지 않는 시스템 — Backoff로 기다리고, Circuit Breaker로 차단하고, Decorator로 깔끔하게 분리한다

S17 마무리 · 다음 세션 예고

M3 · Day 05

오늘 배운 것

- Retry Storm이 왜 위험한지
- Exponential Backoff + Jitter로 재시도 분산
- NON_RETRYABLE 에러 즉시 차단
- Decorator 패턴으로 관심사 분리
- Circuit Breaker 3상태 전이 구현

다음 세션 예고

- S18 — 이벤트 기반 아키텍처
- EventEmitter · Observer 패턴
- 도메인 이벤트 발행 · 구독
- 비동기 워크플로우 설계

숙제: RetryAdapterDecorator에 maxAttempts 파라미터 외부 주입 가능하도록 수정

실습 명령: `npm run exercise:s17:answer` — 모든 테스트 통과 확인